

LC 1814 - Nice Pairs in an Array

Authored by: Arpit Raj, LNMIIT Jaipur • August 26, 2026 ©

Navi • Google • Amazon

Logical Derivation

Any 2 pairs are nice if:

1. $0 \leq i < j < \text{nums.length}$
2. $\text{nums}[i] + \text{rev}(\text{nums}[j]) == \text{nums}[j] + \text{rev}(\text{nums}[i])$

basically $\rightarrow \text{nums}[i] - \text{rev}(\text{nums}[i]) == \text{nums}[j] - \text{rev}(\text{nums}[j])$

Let frequency be f . Because we need pairs, we sum combinations:

$$\Sigma [(f \times (f - 1)) / 2]$$

Examples & Dry Run

Eg 1: [42, 11, 1, 97]

$$42 - 24 = 18$$

$$11 - 11 = 0$$

$$1 - 1 = 0$$

$$97 - 79 = 18$$

Frequencies mapping:

$$18 \rightarrow 2$$

$$0 \rightarrow 2$$

$$\text{ans} = {}^2C_2 + {}^2C_2 = 2$$

Eg 2: [13, 10, 35, 24, 76]

$$13 - 31 = -18$$

$$10 - 1 = 9$$

$$35 - 53 = -18$$

$$24 - 42 = -18$$

$$76 - 67 = 9$$

so -18, -18, -18, 9, 9

$$\text{ans} = (3 \times 2) / 2 + (2 \times 1) / 2 = 3 + 1 = 4$$

C++ Implementation

```
class Solution {
public:
    const int MOD = 1e9+7;

    int revnum(int num) {
        int ans = 0;

        while (num > 0) {
            int dig = num % 10;
            ans = ans * 10 + dig;
            num /= 10;
        }

        return ans;
    }

    int countNicePairs(vector<int>& nums) {
        long long pairs = 0;
        int n = nums.size();
        unordered_map<int, int> freq;

        // basically equation rearrange krke dekho
        // needs nums[i]-rev(nums[i])
        // baad me combinations ke liye sum of f*(f-1)/2

        for (int i = 0; i < n; i++) {
            freq[nums[i] - revnum(nums[i])]++;
        }

        for (auto &it : freq) {
            long long f = it.second;
            pairs += (f * (f - 1)) / 2;
        }

        return pairs % MOD;
    }
};
```